

Capt.N



Session : 2025-29
Department : Computer Science &IT
Field: CS & IT
Course Title : Programming Fundamental
Instructor Name : Dr.Wajid Aziz
Group A
1st Semester Mini Project
Project Name : Banking Managemnt System
Roll Numbers : 61,71,81,91
Students Name
1) Nouman Majeed
2) Abdul Mohiz
3) Aliyan Mughal
4) Raja Areeb

Banking Management System Report

A **Banking Management System** is a software application that helps banks manage customer accounts and financial operations efficiently. It allows account creation, data storage, retrieval, and basic account management in a systematic way.



Tasks Performed by the System

- **Add Account Record** – Create a new bank account with account number, account holder's name, and balance.
- **Display All Accounts** – Show all existing accounts in the system.
- **Search Account Record** – Find and display a specific account using its account number.
- **Exit the System** – Safely close the application.

Algorithm of Banking Management System

Data Structure:

Bank Account:

- accountNumber : int
- firstName : string (or char array)
- lastName : string (or char array)
- balance : float

Main Algorithm:

1. Start Program.
2. Display greeting message and prompt user input for interaction.
3. Loop until user chooses to exit.

1. Display Main Menu:

1. Add Record
2. Display All Records
3. Search Record
4. Exit

2. Read user's choice:

3. Execute corresponding function:

- **Choice 1:** addRecord()
- **Choice 2:** displayRecords()
- **Choice 3:** searchRecord()
- **Choice 4:** Exit program.
- **Invalid choice:** Show error message.

Function: addRecord()

1. Open file BankData.txt in append mode.
2. Prompt user to enter:
 - Account Number
 - First Name
 - Last Name
 - Balance.
3. Write the entered data to the file in one line.
4. Close the file.
5. Display "Record added successfully".

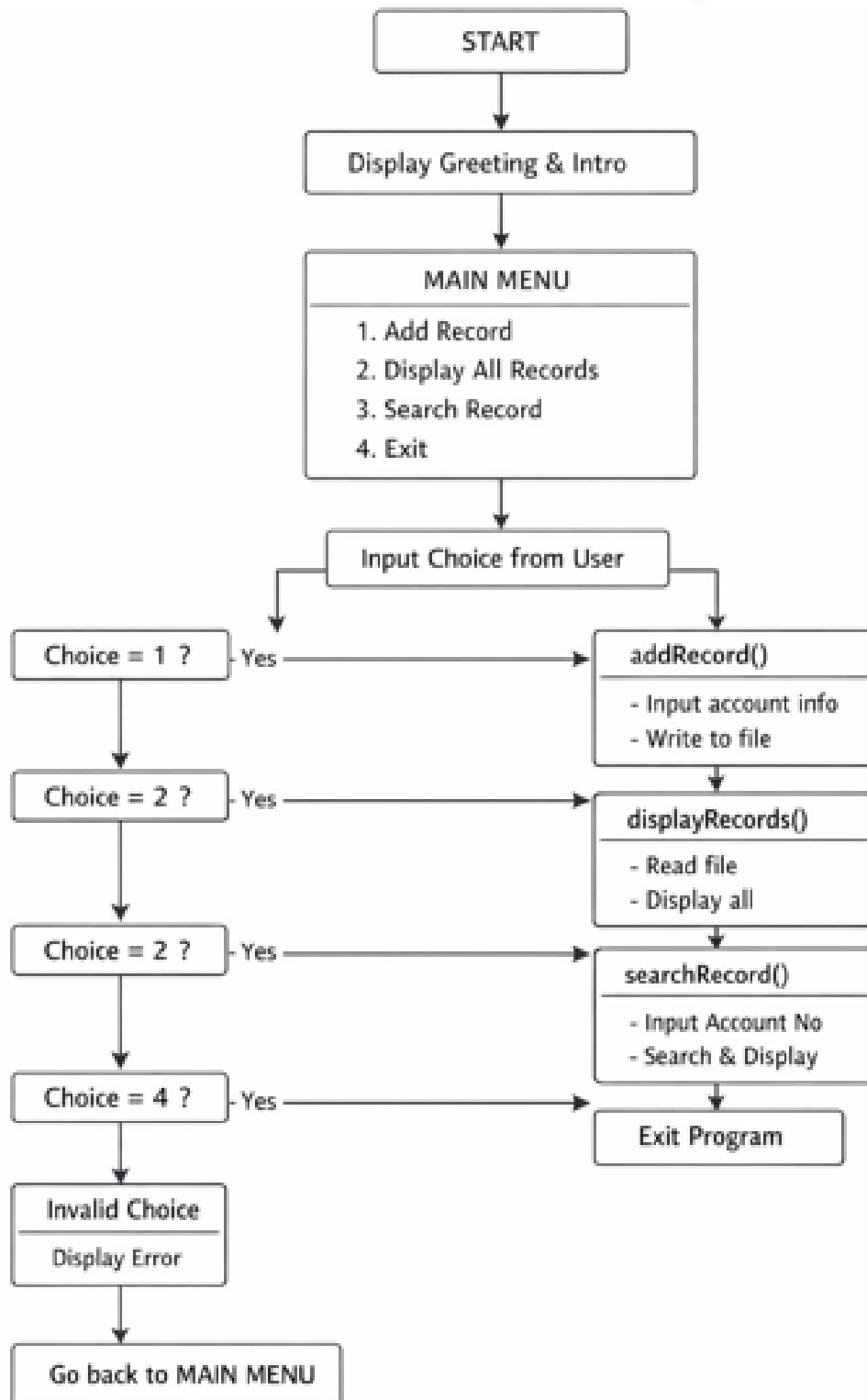
Function: displayRecords()

1. Open file BankData.txt in read mode.
2. If file does not exist, display "No records found" and return.
3. Read file line by line:
 1. Extract accountNumber, firstName, lastName, balance.
 2. Display all details in a formatted way.
4. Close file.

Function: searchRecord()

1. Open file BankData.txt in read mode.
2. If file does not exist, display "File not found" and return.
3. Prompt user to enter Account Number to search.
4. Read file line by line:
 1. Compare each record's accountNumber with the searched number.
 2. If match found:
 1. Display full record.
 2. Stop searching.
5. If no match found, display "Record not found".
6. Close file.

Flow Chart of Banking Management System



Complete C++ Code of banking Management System

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;
```

```
// Structure for Bank Account
struct BankAccount
{
    int accountNumber;
    char firstName[20];
    char lastName[20];
    float balance;
};
```

```
// Function Prototypes
void addRecord();
void displayRecords();
void searchRecord();
```

```
int main()
{
    int choice;
    string a;
```

```
// ===== INTRO SECTION =====
cout << "Hi! Group A Members, How are you?\n";
getline(cin, a); // User can type response
```

```
cout << "\nYour First Semester Mini Project is Following:\n";
cout << "\n===== Banking Management System =====\n";
```

```
// ===== MAIN MENU =====
do
{
    cout << "\n1. Add Record\n";
    cout << "2. Display All Records\n";
    cout << "3. Search Record\n";
    cout << "4. Exit\n";
    cout << "Enter your choice: ";
    cin >> choice;
```

```
    if (choice == 1)
        addRecord();
    else if (choice == 2)
        displayRecords();
```

```
    else if (choice == 3)
        searchRecord();
    else if (choice == 4)
        cout << "Program exited successfully.\n";
    else
        cout << "Invalid choice! Try again.\n";
```

```
    } while (choice != 4);
```

```
    return 0;
}
```

```
// ===== Add Record =====
```

```
void addRecord()
{
    ofstream file("BankData.txt", ios::app);
    BankAccount acc;
```

```
    cout << "\nEnter Account Number: ";
    cin >> acc.accountNumber;
    cout << "Enter First Name: ";
    cin >> acc.firstName;
    cout << "Enter Last Name: ";
    cin >> acc.lastName;
    cout << "Enter Balance: ";
    cin >> acc.balance;
```

```
    file << acc.accountNumber << " "
        << acc.firstName << " "
        << acc.lastName << " "
        << acc.balance << endl;
```

```
    file.close();
    cout << "Record added successfully!\n";
}
```

```
// ===== Display Records =====
```

```
void displayRecords()
{
    ifstream file("BankData.txt");
    BankAccount acc;
    bool found = false;
```

```
    if (!file)
    {
        cout << "No records found!\n";
        return;
    }
```

```
    cout << "\n===== All Bank Records =====\n";
```

```
while (file >> acc.accountNumber >> acc.firstName >> acc.lastName >> acc.balance)
{
    cout << "\nAccount Number: " << acc.accountNumber;
    cout << "\nName: " << acc.firstName << " " << acc.lastName;
    cout << "\nBalance: " << acc.balance << endl;
    found = true;
}
```

```
if (!found)
    cout << "No record available.\n";
```

```
file.close();
}
```

```
// ===== Search Record =====
```

```
void searchRecord()
{
    ifstream file("BankData.txt");
    BankAccount acc;
    int searchAcc;
    bool found = false;
```

```
if (!file)
{
    cout << "File not found!\n";
    return;
}
```

```
cout << "\nEnter Account Number to search: ";
cin >> searchAcc;
```

```
while (file >> acc.accountNumber >> acc.firstName >> acc.lastName >> acc.balance)
{
    if (acc.accountNumber == searchAcc)
    {
        cout << "\nRecord Found!";
        cout << "\nAccount Number: " << acc.accountNumber;
        cout << "\nName: " << acc.firstName << " " << acc.lastName;
        cout << "\nBalance: " << acc.balance << endl;
        found = true;
        break;
    }
}
```

```
if (!found)
    cout << "Record not found!\n";
```

```
file.close();
}
```


Conclusion:

The Banking Management System project successfully demonstrates the implementation of basic banking operations such as adding new accounts, displaying all records, and searching for specific accounts using C++. The project effectively uses **structures** to organize account data, **file handling** to store and retrieve records, and a **menu-driven program** with loops and conditionals to control user interaction. By implementing separate functions for each task, we applied **structured programming principles**, which improved code readability and modularity. This project enhanced our understanding of **data organization**, **logical problem-solving**, and real-world applications of C++ in managing and storing information efficiently.